SOFTWARE REQUIREMENTS SPECIFICATION

for

Remote MCP Control System - Autonomous System Admin

Prepared by :

1. **Tejaswini Durge (22210270)**
2. **Arya Kadam (22210766)**
3. **Soham Deshmukh (22211060)**
4. **Prasad Kute (22210330)**

Submitted to :

Prof. Hanmant Magar

March 20, 2026

Contents

1 Introduction **4**
1.1 Purpose . . . 4
1.2 Scope . . . 4
1.3 Definitions, Acronyms, and Abbreviations . . . 5
1.4 References . . . 5
1.5 System Overview . . . 5

2 Overall Description **6**
2.1 Product Perspective . . . 6
2.2 Product Functions . . . 6
2.2.1 Administrative Core . . . 6
2.2.2 Security Management . . . 7
2.3 User Characteristics . . . 7
2.3.1 End User Profiles . . . 7
2.4 Operating Environment . . . 7
2.4.1 Host Requirements . . . 7
2.5 Design and Implementation Constraints . . . 7

3 Functional Requirements **9**
3.1 Authentication and Security Enforcement . . . 9
3.2 Agent Reasoning & NLP . . . 9
3.3 Core Execution Tools . . . 9
3.3.1 Use Case Diagram . . . 9

4 Non-Functional Requirements **11**
4.1 Security . . . 11
4.2 Performance . . . 11
4.3 Usability & Maintainability . . . 11

5 System Architecture and Interface Requirements **12**
5.1 System Architecture . . . 12
5.1.1 Data Flow Diagram . . . 12
5.2 External Interfaces . . . 12
5.2.1 Communication Interfaces . . . 12

6 Validation and Verification Requirements **14**
6.1 Testing Strategy . . . 14

7 Appendices **15**
7.1 Glossary 15
7.2 System Diagrams 15
7.2.1 Sequence Diagram 15

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document formally defines the operational, technical, and security parameters for the **Remote MCP Control System** - an innovative architecture for autonomous system administration via Large Language Models (LLMs). The document serves three primary objectives:

- **System Blueprint**: Provide a complete architectural specification mapping the natural language inputs to operating system execution.
- **Security Assurance**: Define the strict execution sandboxing and hierarchical access permissions protecting against Arbitrary Code Execution (ACE).
- **Extensibility Framework**: Establish the plugin architecture guidelines utilizing Anthropic's Model Context Protocol (MCP) for future expansions.

1.2 Scope

The Remote MCP Control System revolutionizes direct computer administration through its multi-component architecture bridging stochastic AI reasoning with deterministic host hardware:

Core Features

- **Ubiquitous Interaction Layer**:
  - Telegram Bridge serving as the encrypted, asynchronous mobile frontend.
  - User session context and stateful command memory.
- **Semantic Reasoning Engine**:
  - OpenCode Agent interpreting natural language into serialized JSON-RPC tool calls.
  - Dynamic task breakdown and procedural automation.
- **Secure Execution Host**:
  - FastMCP Server exposing system primitives (File I/O, Shell execution, Python evaluation).
  - Multi-layered OS-primitive sandboxing for malicious command interception.

1.3 Definitions, Acronyms, and Abbreviations

MCP Model Context Protocol (Anthropic's open standard for AI tool integration)

LLM Large Language Model

RCE Remote Code Execution

ACE Arbitrary Code Execution

AST Abstract Syntax Tree

RPC Remote Procedure Call (JSON-RPC 2.0)

RBAC Role-Based Access Control

1.4 References

- **Project Documentation**: `README.md` and `Paper.md`
  Primary architectural context, research methodology, and runtime specifications.
- **Anthropic MCP Spec**: https://docs.anthropic.com/en/docs/mcp/
  Official documentation defining the JSON-RPC integration layer used by FastMCP.
- **Telegram Bot API**: https://core.telegram.org/bots/api
  Transport layer documentation defining the asynchronous webhooks and client state management.

1.5 System Overview

The Remote MCP Control System acts as a deterministic, secure gatekeeper that receives natural language from authenticated users, translates it via an LLM into explicit tool-calling schemas, and executes those commands locally within a rigidly confined sandbox.

2 Overall Description

2.1 Product Perspective

The system employs a multi-service pipelined architecture designed to cleanly decouple user UI, LLM reasoning, and host execution:

- **Frontend Layer**:
  - Telegram Bot API providing interaction modalities.
  - Context-aware session management parsing raw shortcuts vs semantic dialogue.
- **Core Logic Layer**:
  - OpenCode Agent constructing structured prompts and evaluating MCP resource availability.
  - Context memory window tracking historical decisions.
- **Execution / Data Layer**:
  - Local FastMCP instance functioning as a resource server.
  - Python subprocess and `exec()` modules functioning under AST-filtered namespaces.

2.2 Product Functions

2.2.1 Administrative Core

- **Dynamic Shell Operations**: Captures `stdout`/`stderr` to allow LLM agents to verify their own actions iteratively.
- **Intelligent File Operations**: Supports absolute/relative path resolution, file reading, and secure writing without arbitrary directory traversal vulnerabilities.
- **Sandboxed Python Automation**: Accepts script payloads from the LLM, filters out malicious network/OS imports via AST parsing, and executes dynamic automation locally.

2.2.2 Security Management

- **Cryptographic Identity**: All connections are rejected unless authenticated against a master `permissions.yaml` utilizing hardcoded Telegram User IDs.
- **Granular Roles**:
  - `Readonly`: Restricted to system telemetry and file consumption.
  - `User`: Permitted to execute safe logic flows without root privileges.
  - `Admin`: Unrestricted system modification and shell access.

2.3 User Characteristics

2.3.1 End User Profiles

- **System Administrators**:
  - Highly technical profiling requiring rapid, mobile-friendly responses to server incidents.
  - Heavy utilization of the `admin` permission class.
- **General Users**:
  - Users who want to automate tedious OS tasks ("Group my downloads by file type") without knowing shell scripting.

2.4 Operating Environment

2.4.1 Host Requirements

- **Operating System**: Windows 10/11 natively supported; POSIX-compliant layers available.
- **Runtime**: Python 3.10+ (requires native `asyncio`).
- **Dependencies**: `fastmcp`, `python-telegram-bot`, `psutil`.
- **Network**: Must maintain an outbound secure HTTPS tunnel to Telegram's REST APIs; no inbound ports (e.g., 80/443) are required to be exposed.

2.5 Design and Implementation Constraints

- **Performance**: The system must acknowledge user interactions in under 1 second (e.g., "Thinking...") to comply with Telegram's UX latency standards.
- **Security**: Due to the lack of hardware virtualization (MicroVMs), safety relies heavily on blacklists and Abstract Syntax Tree (AST) scanning. Any bypass by an LLM hallucination represents a critical vulnerability.

- **Coupling**: The architecture strictly mandates the use of Anthropic's standard Schema specifications to ensure the system doesn't break when switching downstream LLM providers.

3 Functional Requirements

3.1 Authentication and Security Enforcement

- **FR1**: The system must extract the Telegram user's ID on every incoming payload and validate it against the `config/permissions.yaml` file.
- **FR2**: Unregistered payloads must be silently dropped or return a generic "Unauthorized" message to prevent endpoint enumeration.
- **FR3**: The FastMCP server must enforce Role-Based Access Control (RBAC) prior to executing any subprocess.

3.2 Agent Reasoning & NLP

- **FR4**: The `OpenCodeAgent` must maintain a conversational memory limit (e.g., maximum 20 messages) to prevent context window exhaustion.
- **FR5**: The agent must parse vague natural language into explicit JSON-RPC payloads conformant to the MCP specification.
- **FR6**: The agent must interpret failure streams (`stderr`) from the host and attempt autonomous self-correction without immediate user intervention.

3.3 Core Execution Tools

- **FR7**: The `ShellTool` must execute non-interactive commands within a maximum timeout of 30 seconds to prevent resource exhaustion (DoS).
- **FR8**: The `FilesTool` must restrict operations to authorized absolute directories, preventing arbitrary traversal (e.g., accessing `C:/Windows/System32` illegally).
- **FR9**: The `PythonExecTool` must parse execution strings utilizing Python's native `ast` module and flag node types attempting imports of `os`, `sys`, or `subprocess`.

3.3.1 Use Case Diagram

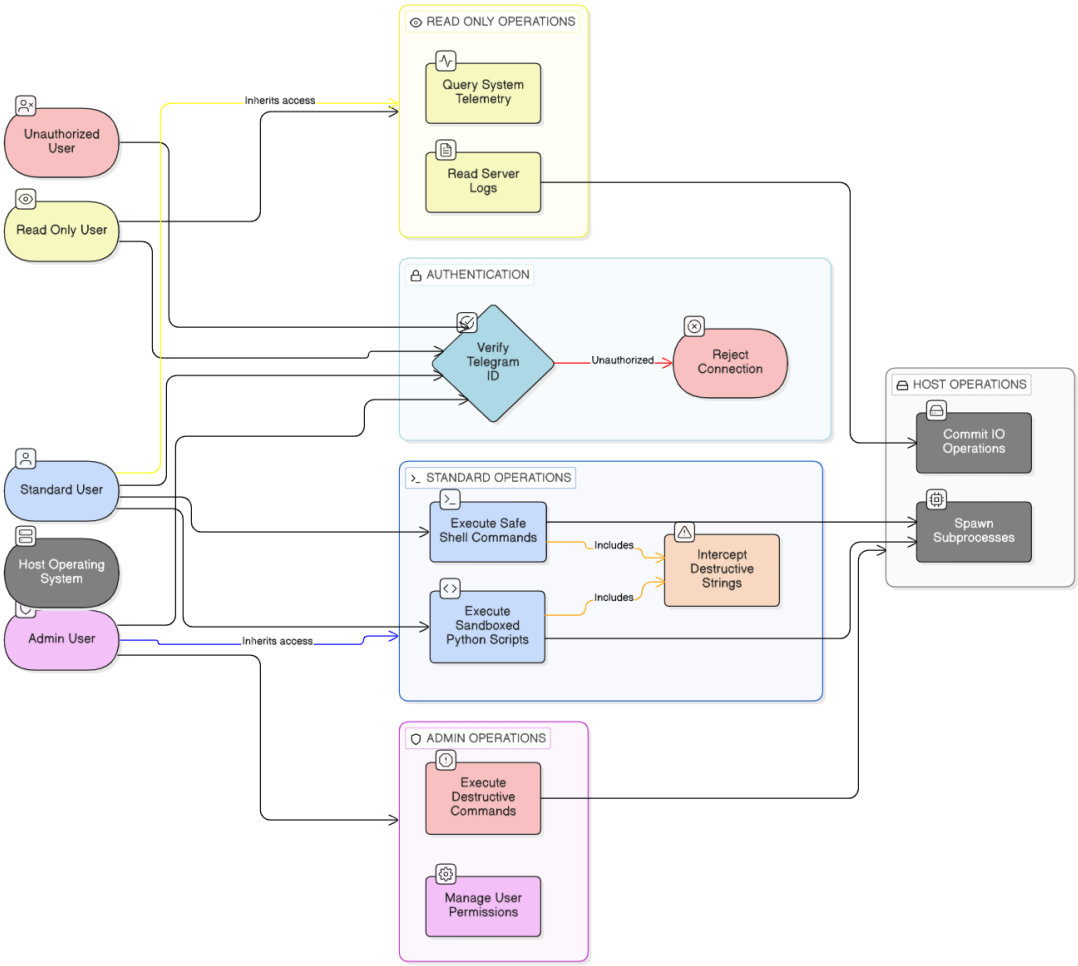

Figure 3.1: System Use Case Diagram

4 Non-Functional Requirements

4.1 Security

- **NFR1**: The Telegram API Token and local authentication passwords must never be logged or hardcoded, residing exclusively in `.env` dotfiles.
- **NFR2**: The Sandbox primitive must evaluate destructive shell commands prior to thread instantiation; failure to catch a destructive string must result in system soft-lock rather than execution sequence continuation.

4.2 Performance

- **NFR3**: Direct deterministic tool calls routing around the LLM agent (e.g., `/shell ping 8.8.8.8`) must execute and return to the UI in less than 800 milliseconds.
- **NFR4**: The FastMCP server process threading must not consume more than 2% of idle CPU state.

4.3 Usability & Maintainability

- **NFR5**: The system architecture must allow developers to drag-and-drop new subclasses into the `plugins/` directory without modifying the central bootloops.
- **NFR6**: The UI must format all terminal outputs inside Telegram markdown blocks for maximal readability on mobile form factors.

5 System Architecture and Interface Requirements

5.1 System Architecture

The Remote MCP Control System effectively utilizes an event-driven Pipeline Architecture:

- **Source**: User Chat Interface (Telegram Bridge routing).
- **Processor**: Language Model context builder (OpenCode Agent JSON-RPC framing).
- **Sink**: FastMCP execution server committing writes to the local Host OS.

5.1.1 Data Flow Diagram

5.2 External Interfaces

5.2.1 Communication Interfaces

- Secure HTTPS transport bound exclusively utilized for downstream Telegram API communications.
- Locally bound sockets (e.g., `localhost:8765`) used for inter-process communication between the Agent Client and the MCP Server, guaranteeing the execution boundary cannot be compromised externally.

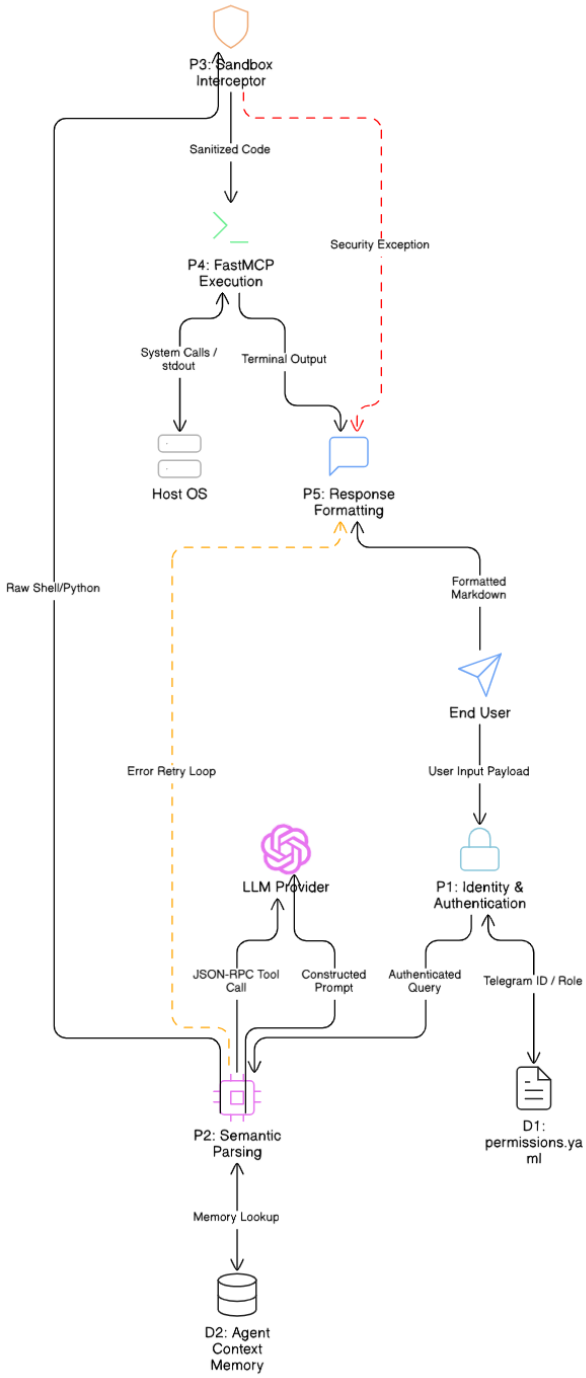

Figure 5.1: Data Flow Diagram

6 Validation and Verification Requirements

6.1 Testing Strategy

- **Red-Team Penetration**: Simulating prompt-injection attacks directly into the OpenCode Agent to evaluate the efficacy of the `Sandbox` interceptors.
- **Unit Testing**: Ensuring AST parsing correctly identifies heavily obfuscated, implicitly nested import statements.
- **End-to-End Validation**: User testing the system with highly vague prompts to confirm the LLM reliably self-corrects based on recursive `stderr` logging.

7 Appendices

7.1 Glossary

- **MCP**: Model Context Protocol.
- **AST**: Abstract Syntax Tree.
- **JSON-RPC**: A remote procedure call protocol encoded in JSON.
- **Sandboxing**: A security mechanism separating running programs, often used to execute untested or untrusted code locally.

7.2 System Diagrams

7.2.1 Sequence Diagram

The system architecture and workflows are visualized through the following diagrams, which provide both high-level overviews and detailed operational perspectives:

- **Figure ??**: High-Level System Architecture Diagram
  Details the isolation boundary between the Telegram API, the Agent semantic layer, and the local host operating system.
- **Figure 3.1**: Use Case Diagram
  Captures user-system interactions depending strictly on the RBAC `permissions.yaml` assigned roles (ReadOnly, User, Admin).
- **Figure 5.1**: Data Flow Diagram
  Demonstrates the temporal progression of a natural language string converting into structured execution instructions via the Data and Transport layers of the MCP protocol.
- **Figure 7.1**: Sequence Diagram
  Illustrates the precise operational lockstep occurring within the `Sandbox` intercept module during an attempted Arbitrary Code Execution event.

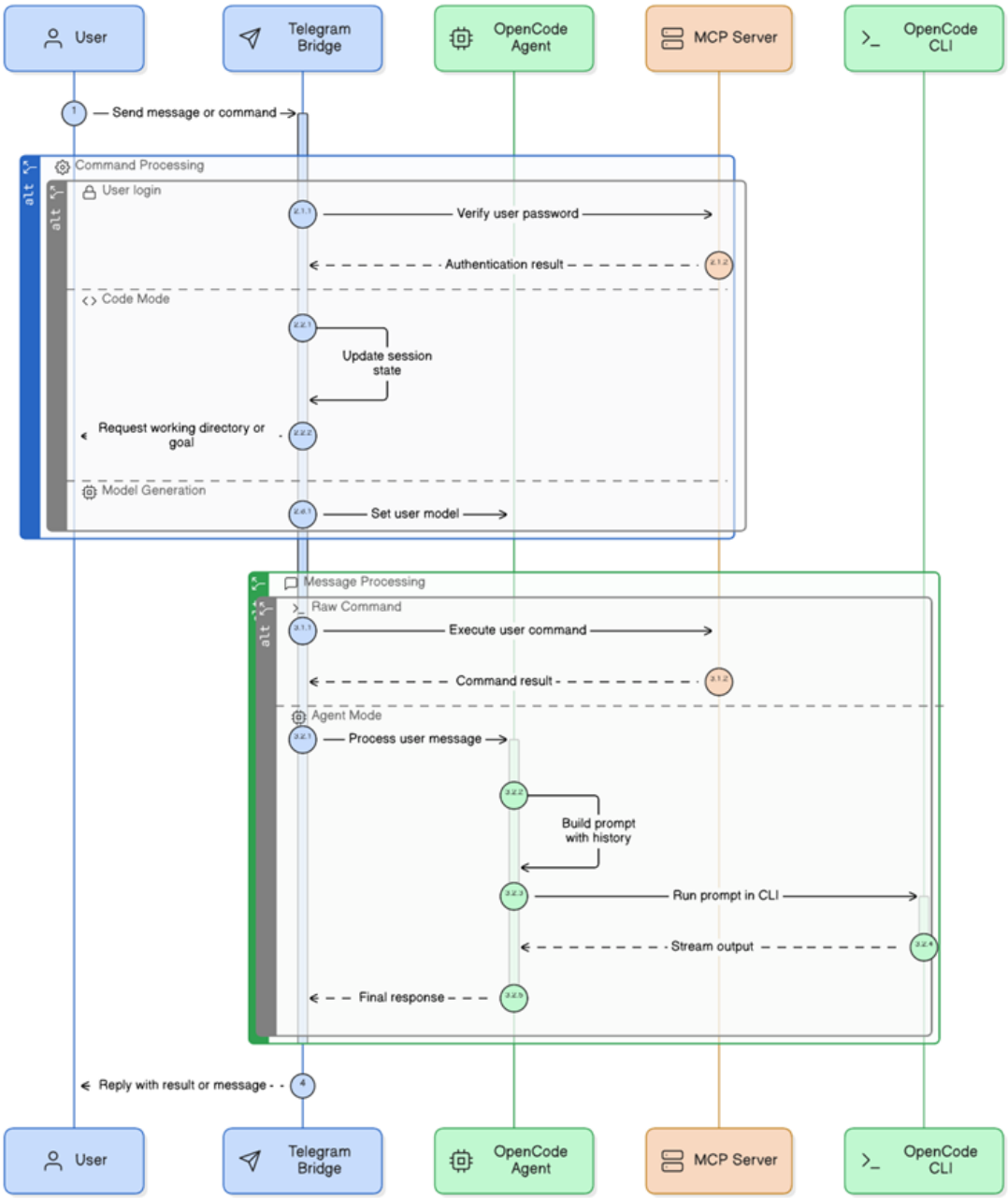

Figure 7.1: Sequence Diagram